Heart Disease Prediction MLOps Project Report

1. Objective

This project implements an end-to-end MLOps pipeline for the UCI Heart Disease dataset. The goal is to build a reproducible classifier that predicts risk of heart disease and serve it through a containerized API.

2. Dataset

The dataset contains medical attributes such as age, sex, chest pain type, cholesterol, blood pressure, and exercise-induced angina. The target is the binary heart-disease indicator.

3. Data Cleaning and EDA

The dataset was downloaded from the UCI repository, missing values were handled, and the target variable was converted to binary labels. The cleaned data is stored in data/heart_cleaned.csv. Summary statistics and plots were generated for class balance, age distribution, and correlation analysis.

4. Feature Engineering

The feature schema includes the 13 numerical input attributes used by the classifier, with consistent column ordering preserved for training and inference. This ensures the same preprocessing pipeline is used for deployment.

5. Model Development

Two models were trained: Logistic Regression and Random Forest. These were compared using accuracy, precision, recall, and ROC-AUC. The best model selected for deployment is Logistic Regression.

6. Experiment Tracking

MLflow was configured for training runs. Parameters, metrics, and model artifacts were logged for each run to enable reproducibility and comparison of different experiments.

7. Packaging and Reproducibility

The final model is serialized with joblib and saved under artifacts/heart_disease_model.joblib. The project includes a clean dependency file and reproducible data pipeline scripts.

8. API and Containerization

A FastAPI application exposes a POST /predict endpoint that accepts patient attributes and returns the prediction, risk label, and confidence. The service is packaged in Docker and runs locally with uvicorn.

9. CI/CD and Deployment

The repository includes a GitHub Actions workflow that installs dependencies, runs tests, and performs a training step. A Kubernetes deployment manifest is also included for local deployment and scaling.

10. Monitoring and Reporting

The API exposes Prometheus-compatible metrics at /metrics and logs each request. This makes the model service observable and suitable for production-style monitoring workflows. The screenshots and generated artifacts are saved in the screenshots directory and used for reporting.

Model performance summary

- Best model: Logistic Regression
- Accuracy: 0.8667
- Precision: 0.8846
- Recall: 0.8214
- ROC-AUC: 0.9397

Conclusion

This assignment demonstrates a full MLOps cycle: data ingestion, cleaning, EDA, feature engineering, model selection, training, experiment tracking, packaging, API deployment, monitoring, and documentation. The resulting system is reproducible and ready for further cloud deployment and production hardening.